

Tema 4.1. Composición

1. En C, podemos crear estructuras mayores **componiendo** unas con otras, que suelen describirse como "A tiene-un/tiene-varios B". Pon un ejemplo, empleando **struct**, de una línea de puntos, donde puntos tienen dos coordenadas (**x** e **y**), y la línea esta hecha de dos puntos. Incluye una función para calcular la distancia entre puntos y otra para hallar la longitud de una línea.

Respuesta

En C, la composición se puede ver cuando una struct contiene otras struct. En este caso, una línea "tiene dos puntos", y cada punto "tiene dos coordenadas". La relación se expresa metiendo variables de tipo Punto dentro de la estructura Linea.

```
#include <stdio.h> #include <math.h>

typedef struct { double x; double y; } Punto;

typedef struct { Punto origen; Punto destino; } Linea;

double distancia(Punto p1, Punto p2) { double dx = p2.x - p1.x; double dy = p2.y - p1.y; return sqrt(dx * dx + dy * dy); }

double longitud(Linea l) { return distancia(l.origen, l.destino); }

int main() { Punto p1 = {0, 0}; Punto p2 = {3, 4};

    Linea l = {p1, p2};

    printf("Longitud: %.2f\n", longitud(l));

    return 0;

}
```

Aquí Linea está compuesta por dos variables de tipo Punto. A su vez, cada Punto está compuesto por dos coordenadas numéricas, x e y.

2. Ahora transforma ese ejemplo a orientación a objetos con Java, para tener un primer ejemplo de **composición** en orientación a objetos. Crea una clase **Punto**, y una clase **Linea**. La clase **Punto** debe tener un método para calcular distancia a otro **Punto** y **Linea** debe tener un método para calcular su longitud. Gracias a la ocultación de información, supera a C, garantizando que los puntos sean inmutables, al igual que la línea, que una vez creada, no queremos que se modifique de qué a qué puntos va dicha línea.

Respuesta

En Java, la composición se representa creando una clase que contiene objetos de otra clase como atributos. En este caso, la clase Linea contiene dos objetos de la clase Punto.

Además, Java permite mejorar el diseño respecto a C mediante encapsulación. Si los atributos son private final y no existen métodos que los modifiquen, los objetos se vuelven inmutables.

```
public final class Punto { private final double x; private final double y;
```

```
    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double distanciaA(Punto otro) {
        double dx = otro.x - this.x;
        double dy = otro.y - this.y;
        return Math.sqrt(dx * dx + dy * dy);
    }

    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }
}
```

```
}
```

```
public final class Linea { private final Punto origen; private final Punto destino;
```

```
    public Linea(Punto origen, Punto destino) {
        if (origen == null || destino == null) {
            throw new IllegalArgumentException("Los puntos no pueden ser null");
        }

        this.origen = origen;
        this.destino = destino;
    }

    public double longitud() {
        return origen.distanciaA(destino);
    }

    public Punto getOrigen() {
        return origen;
    }
}
```

```
public Punto getDestino() {  
    return destino;  
}
```

```
}
```

Como Punto es inmutable, no hay peligro en devolverlo directamente desde Linea. Una vez creada una línea, no se puede cambiar su origen ni su destino.

3. ¿Qué significa la **multiplicidad** en la composición? En el ejemplo anterior, ¿cuál es la multiplicidad entre **Linea** y **Punto**? Indícalo expresando la multiplicidad en ambas direcciones, de **Linea** a **Punto** y de **Punto** a **Linea**.

Respuesta

La multiplicidad indica cuántos objetos de una clase pueden estar relacionados con objetos de otra clase. Es decir, sirve para expresar relaciones del tipo "una línea tiene exactamente dos puntos" o "un punto puede formar parte de muchas líneas".

En el ejemplo anterior, desde Linea hacia Punto, la multiplicidad es exactamente 2, porque una línea tiene un punto de origen y un punto de destino.

Desde Punto hacia Linea, la multiplicidad puede ser 0..*, porque un punto podría no estar en ninguna línea, o podría usarse en muchas líneas distintas.

4. ¿Qué significa composición **fuerte** y composición **débil**? ¿Qué consecuencia implica en relación al ciclo de vida de los objetos? Indica a cuál solemos referirnos como "**asociación o agregación**" y a cuál como "**composición**" propiamente.

Respuesta

La composición débil se da cuando un objeto contiene o referencia a otro, pero el objeto contenido puede existir independientemente. Por ejemplo, un departamento puede tener profesores, pero esos profesores no desaparecen si desaparece el departamento.

La composición fuerte se da cuando el objeto contenido depende del objeto contenedor para existir. Es decir, su ciclo de vida está ligado al del objeto principal. Por ejemplo, si una línea crea internamente sus puntos y estos no se comparten, se puede entender que los puntos pertenecen solo a esa línea.

Normalmente, la composición débil se relaciona con la asociación o agregación. La composición fuerte es la que suele llamarse composición propiamente dicha.

5. Cuando una clase usa a otra al recibirla o devolverla como parámetro en algún método, al hacer **new** dentro de un método, o al usarlas como variables locales, ¿hablamos de composición o de "**dependencia**"?

Respuesta

En ese caso se habla de dependencia, no de composición. Una clase depende de otra cuando la usa temporalmente en un método, por ejemplo como parámetro, como valor devuelto, como variable local o creando un objeto con new dentro de un método.

La diferencia principal es que en la composición una clase guarda a otra como atributo. En cambio, en la dependencia el uso es puntual y no queda como parte permanente del estado del objeto.

Por ejemplo, si una clase Calculadora recibe un Punto solo para hacer una operación, hay dependencia. Pero si una Linea guarda dos Punto como atributos, hay composición.

6. En el ejemplo anterior de línea y punto, programa la relación entre **Linea** y **Punto** de dos formas. Una **como composición fuerte**, donde el ciclo de vida de los puntos está ligado al de Linea y otra **como composición débil**, donde no.

Respuesta

Una composición fuerte puede implementarse haciendo que Linea cree internamente sus puntos. Desde fuera no se le pasan objetos Punto, sino coordenadas. Así, los puntos pertenecen a la línea.

```
public final class LineaFuerte { private final Punto origen; private final Punto destino;
```

```
    public LineaFuerte(double x1, double y1, double x2, double y2) {
        this.origen = new Punto(x1, y1);
        this.destino = new Punto(x2, y2);
    }

    public double longitud() {
        return origen.distanciaA(destino);
    }
}
```

En cambio, una composición débil se da cuando los puntos se crean fuera y se pasan a la línea. En ese caso, los mismos puntos podrían ser compartidos por varias líneas.

```
public final class LineaDebil { private final Punto origen; private final Punto destino;
```

```
    public LineaDebil(Punto origen, Punto destino) {
        if (origen == null || destino == null) {
            throw new IllegalArgumentException("Los puntos no pueden ser null");
        }

        this.origen = origen;
        this.destino = destino;
    }

    public double longitud() {
```

```
        return origen.distanciaA(destino);  
    }  
}
```

La diferencia no está solo en el código, sino en la idea de diseño: en la fuerte, los puntos forman parte interna de la línea; en la débil, la línea usa puntos que existen fuera.

7. En Java, en la composición fuerte, ¿cuando el contenedor destruye los objetos? No se observa que **Línea** destruya los **Punto** explícitamente, ¿Por qué?

Respuesta

En Java, el contenedor no destruye explícitamente los objetos contenidos. Esto ocurre porque Java usa un recolector de basura, conocido como garbage collector.

Cuando un objeto deja de tener referencias accesibles, Java puede eliminarlo automáticamente de memoria. Por eso no se escribe algo como delete, como sí podría hacerse en C++.

En una composición fuerte, cuando la Línea deja de usarse y sus Punto no están referenciados desde ningún otro sitio, esos puntos también quedan disponibles para ser eliminados por el recolector de basura.

8. Pon un ejemplo de composicion débil entre un departamento que tiene varios profesores. Implementa dos composiciones a la vez: entre el departamento y todos sus profesores y entre el departamento y su director, que es un profesor del departamento. Siempre debe haber un director en el departamento desde el inicio. Lanza excepciones si se viola la invariante. Emplea arrays primitivos de Java, estilo **Profesor[]**, con máximo 50, pero no rompas la encapsulación, no desveles que estás empleando un array, permite añadir un **Profesor** al final de la lista, y eliminar un profesor dada su posición. Da acceso a los profesores con un método para saber cuántos hay y otro para obtener un profesor por posición. El director se puede cambiar por otro profesor del departamento. Sin embargo, ten en cuenta esta invariante de clase: el director debe formar siempre parte de la lista de profesores, es decir, ten cuidado al cambiar el director o al eliminar un profesor.

Respuesta

En este ejemplo hay composición débil porque los profesores existen fuera del departamento. El departamento solo guarda referencias a ellos. Además, hay dos relaciones a la vez: el departamento tiene varios profesores y tiene un director, que debe ser uno de esos profesores.

La invariante principal es que siempre debe haber un director y ese director debe estar dentro de la lista de profesores. Por eso, al crear el departamento, cambiar el director o eliminar profesores, deben hacerse comprobaciones.

```
public class Profesor { private final String nombre;
```

```
public Profesor(String nombre) {
    if (nombre == null || nombre.isBlank()) {
        throw new IllegalArgumentException("El nombre no puede estar vacío");
    }

    this.nombre = nombre;
}

public String getNombre() {
    return nombre;
}
```

```
}
```

```
public class Departamento { private static final int MAX_PROFESORES = 50;
```

```
    private final String nombre;
    private final Profesor[] profesores;
    private int numProfesores;
    private Profesor director;

    public Departamento(String nombre, Profesor director) {
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre no puede estar vacío");
        }

        if (director == null) {
            throw new IllegalArgumentException("Debe existir un director inicial");
        }

        this.nombre = nombre;
        this.profesores = new Profesor[MAX_PROFESORES];
        this.profesores[0] = director;
        this.numProfesores = 1;
        this.director = director;
    }

    public void addProfesor(Profesor profesor) {
        if (profesor == null) {
            throw new IllegalArgumentException("El profesor no puede ser null");
        }

        if (numProfesores == MAX_PROFESORES) {
            throw new IllegalStateException("No caben más profesores");
        }

        profesores[numProfesores] = profesor;
        numProfesores++;
    }
}
```

```
public void removeProfesor(int pos) {
    comprobarPosicion(pos);

    Profesor eliminado = profesores[pos];

    if (eliminado == director) {
        throw new IllegalStateException("No se puede eliminar al director");
    }

    for (int i = pos; i < numProfesores - 1; i++) {
        profesores[i] = profesores[i + 1];
    }

    profesores[numProfesores - 1] = null;
    numProfesores--;
}

public int getNumProfesores() {
    return numProfesores;
}

public Profesor getProfesor(int pos) {
    comprobarPosicion(pos);
    return profesores[pos];
}

public Profesor getDirector() {
    return director;
}

public void setDirector(Profesor nuevoDirector) {
    if (nuevoDirector == null) {
        throw new IllegalArgumentException("El director no puede ser null");
    }

    if (!contieneProfesor(nuevoDirector)) {
        throw new IllegalArgumentException("El director debe pertenecer al departamento");
    }

    this.director = nuevoDirector;
}

private boolean contieneProfesor(Profesor profesor) {
    for (int i = 0; i < numProfesores; i++) {
        if (profesores[i] == profesor) {
            return true;
        }
    }

    return false;
}

private void comprobarPosicion(int pos) {
```

```

        if (pos < 0 || pos >= numProfesores) {
            throw new IndexOutOfBoundsException("Posición incorrecta");
        }
    }
}

```

```

}

```

No se devuelve el array interno, por lo que no se rompe la encapsulación. Desde fuera solo se puede saber cuántos profesores hay y obtener uno por posición.

9. En Java, existen también **List**, cambia y muestra cómo sería el código anterior empleando **List** en vez de arrays primitivos. ¿Qué parte del código original te has ahorrado? Además, fíjate en el método **getProfesor(int pos)**: si en su lugar existiera un método que devolviera todos los profesores a la vez, ¿qué problema tendría devolver directamente la lista interna? ¿Cómo lo resolverías?

Respuesta

Con List, el código se simplifica porque ya no es necesario controlar manualmente el tamaño máximo, desplazar elementos al eliminar ni guardar una variable numProfesores. La propia lista se encarga de almacenar, añadir, eliminar y consultar elementos.

```
import java.util.ArrayList; import java.util.Collections; import java.util.List;
```

```
public class DepartamentoConList { private final String nombre; private final List profesores; private Profesor director;
```

```

    public DepartamentoConList(String nombre, Profesor director) {
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre no puede estar vacío");
        }

        if (director == null) {
            throw new IllegalArgumentException("Debe existir un director inicial");
        }

        this.nombre = nombre;
        this.profesores = new ArrayList<>();
        this.profesores.add(director);
        this.director = director;
    }

    public void addProfesor(Profesor profesor) {
        if (profesor == null) {
            throw new IllegalArgumentException("El profesor no puede ser null");
        }

        profesores.add(profesor);
    }
}

```

```

public void removeProfesor(int pos) {
    Profesor eliminado = profesores.get(pos);

    if (eliminado == director) {
        throw new IllegalStateException("No se puede eliminar al director");
    }

    profesores.remove(pos);
}

public int getNumProfesores() {
    return profesores.size();
}

public Profesor getProfesor(int pos) {
    return profesores.get(pos);
}

public Profesor getDirector() {
    return director;
}

public void setDirector(Profesor nuevoDirector) {
    if (nuevoDirector == null) {
        throw new IllegalArgumentException("El director no puede ser null");
    }

    if (!profesores.contains(nuevoDirector)) {
        throw new IllegalArgumentException("El director debe pertenecer al
departamento");
    }

    this.director = nuevoDirector;
}

public List<Profesor> getProfesores() {
    return Collections.unmodifiableList(profesores);
}

```

```

}

```

Con List se ha ahorrado el array primitivo, la constante de capacidad máxima, el contador manual y el desplazamiento de elementos al eliminar.

Si se devolviera directamente la lista interna, desde fuera se podría modificar el departamento sin pasar por sus métodos. Por ejemplo, se podría eliminar al director y romper la invariante. Para evitarlo, se devuelve una vista no modificable con Collections.unmodifiableList.

10. Al igual que ocurre con las excepciones en Java, que pueden encerrar causas (que son excepciones), de forma recursiva, suponen un tipo especial de composiciones, denominadas composiciones recursivas. Pon

un ejemplo en Java de una **Persona**, que sea inmutable, y que tiene una madre, que es otra **Persona**. Haz un main con un ejemplo de uso con una familia de personas, desde el nieto hasta la abuela. Enumera algún otro ejemplo clásico de composiciones recursivas.

Respuesta

Una composición recursiva aparece cuando una clase contiene una referencia a otro objeto de su mismo tipo. En este caso, una Persona puede tener una madre, que también es una Persona.

La clase puede hacerse inmutable usando atributos private final y no proporcionando métodos modificadores. La madre puede ser null si no se conoce o si se quiere cortar la cadena familiar.

```
public final class Persona { private final String nombre; private final Persona madre;
```

```
    public Persona(String nombre, Persona madre) {
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre no puede estar vacío");
        }

        this.nombre = nombre;
        this.madre = madre;
    }

    public String getNombre() {
        return nombre;
    }

    public Persona getMadre() {
        return madre;
    }
}
```

```
}

public class Main { public static void main(String[] args) { Persona abuela = new Persona("Carmen", null);
Persona madre = new Persona("Laura", abuela); Persona nieto = new Persona("Daniel", madre);
```

```
    System.out.println(nieto.getNombre());
    System.out.println(nieto.getMadre().getNombre());
    System.out.println(nieto.getMadre().getMadre().getNombre());
}
```

```
}
```

Otros ejemplos clásicos de composición recursiva son carpetas que contienen carpetas, nodos de una lista enlazada, árboles genealógicos, árboles de directorios y estructuras de datos como árboles binarios.

11. ¿Qué son las relaciones de composición "bidireccionales"? ¿Qué habría que hacer para implementar este tipo de relación en el ejemplo de **Profesor** y **Departamento**?

Respuesta

Una relación bidireccional es aquella en la que los dos objetos se conocen entre sí. Por ejemplo, un Departamento conoce a sus Profesores, pero cada Profesor también conoce a su Departamento. Para implementarlo, no bastaría con guardar profesores dentro del departamento. También habría que añadir en Profesor un atributo de tipo Departamento. Además, habría que mantener la coherencia en ambos lados. Al añadir un profesor al departamento, también se debería actualizar el departamento del profesor. Al eliminarlo, habría que quitar esa referencia. Esto complica el código porque se deben evitar inconsistencias entre los dos objetos.